



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design and Implementation of an Efficient Priority-Based Emergency Patient Management System using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0307- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTERSCIENCE AND BIOSCIENCES

Submitted by

G. Ramya (192572073)

R. Varshini (192525349)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

Particular	Details
Department:	Computer Science and Engineering – Artificial Intelligence
Programme:	B.E. Computer Science and Engineering
Course Code & Course Name	CSA0307&Data Structures
Academic Year / Batch	2025
Faculty Name	Dr.Kiruthika
Assignment Title	Design And Implementation Of An Efficient Priority-Based Emergency Patient Management System Using Appropriate Data Structures For Severity-Based Treatment And Arrival-Order Processing
Date of Issue	01-09-2026
Date of Submission	01-09-2026
Maximum Marks	100
Course Outcome(s) – CO	
Bloom's Taxonomy Level	
SDG Mapping (SDG 1-17)	SDG 3 – Good Health and Well-Being
Industry / Societal Relevance	Hospital emergency departments require efficient patient prioritization and timely treatment.

B. ASSIGNMENT PROBLEM / CHALLENGE

The assignment focuses on designing and implementing a data-structure-based emergency patient management system for a hospital.

A hospital emergency department receives patients continuously throughout the day. Each patient may require a different level of medical attention depending on the severity of the condition. The system must maintain important information such as Patient ID, arrival time, severity level, and assigned doctor.

Patients with critical or life-threatening conditions must be given higher priority and treated before patients with less severe conditions. However, when two or more patients have the same severity level, they should generally be treated according to their arrival order using

the First-In-First-Out (FIFO) principle.

The system must support patient insertion, priority assignment, identification of the next patient, deletion of treated patients, searching, and updating patient or doctor information.

Different data structures such as a simple queue, array, linked list, and priority queue must be considered. The most appropriate data structure must be selected and justified based on priority-based treatment and arrival-order processing.

C. PROBLEM STATEMENT

A hospital emergency department receives patients continuously throughout the day, and each patient may require a different level of medical attention depending on the severity of their condition.

For every patient, the system maintains:

- Unique Patient ID
- Arrival Time
- Severity Level
- Assigned Doctor
- Arrival Number

Patients with critical or life-threatening conditions must be given higher priority and treated before patients with less severe conditions. When patients have the same severity level, they must generally be treated according to their arrival order.

A conventional FIFO queue is not sufficient because it processes patients only according to arrival time. Therefore, a priority-based data structure is required.

The proposed system uses a **Priority Queue** to manage emergency patients. Higher severity patients are treated first, while patients having equal severity are processed using FIFO order.

D. REQUIREMENTS AND CONSTRAINTS

D.1 Functional Requirements

The system should provide the following functions:

1. Register a new patient.
2. Store Patient ID.
3. Store arrival time.
4. Assign severity level.
5. Store assigned doctor.
6. Assign an arrival number.
7. Insert the patient into the priority queue.
8. Display waiting patients according to priority.
9. Identify the next patient for treatment.
10. Treat and remove the highest-priority patient.
11. Search for a patient using Patient ID.
12. Update patient information.
13. Update severity level.
14. Update assigned doctor.
15. Exit the system safely.

D.2 Severity Levels

Severity	Priority	Description
5	Highest	Critical / Life-threatening
4	High	Very serious
3	Medium	Serious
2	Low	Less serious
1	Lowest	Non-critical

A larger severity value indicates a higher treatment priority.

D.3 Technical Requirements

- Programming Language: C
- Data Structure: Priority Queue
- Implementation: Array-based
- Compiler: GCC / MinGW
- IDE: Visual Studio Code / Code::Blocks / Dev-C++
- Operating System: Windows/Linux/macOS

D.4 Constraints

- Patient ID should be unique.
- Severity should be between 1 and 5.
- The number of patients is limited by the available array size in the current implementation.
- Patients with equal severity must maintain arrival order.
- Patient information must be updated carefully to avoid incorrect records.

D.5 Safety and Reliability

Since the application represents an emergency department, priority handling must be consistent. Critical patients should not be incorrectly placed behind low-priority patients. The system should also maintain accurate patient and doctor information.

E. STUDENT WORK

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 What is the Problem?

The problem is to efficiently manage patients arriving at a hospital emergency department when patients have different levels of medical severity.

A normal queue follows the arrival order:

Patient A → Patient B → Patient C

However, emergency treatment cannot always follow arrival order. A patient arriving later may have a critical condition and therefore needs immediate attention.

For example:

Patient A arrives first with severity 2.

Patient B arrives later with severity 5.

A normal FIFO queue would process Patient A first. However, the proposed system processes Patient B first because the patient has a more severe condition.

Therefore, the system requires a priority-based approach.

1.2 Expected Outcomes

The expected outcomes are:

- Efficient patient registration.
- Correct priority assignment.
- Faster identification of critical patients.
- FIFO processing among patients with equal severity.
- Correct removal of treated patients.
- Efficient searching and updating.
- Reduced possibility of critical patients waiting behind less severe patients.
- Improved emergency department responsiveness.

1.3 Available Information

For every patient, the following information is available:

Information	Purpose
Patient ID	Uniquely identifies the patient
Arrival Time	Records when the patient arrived
Severity	Determines treatment priority
Assigned Doctor	Identifies the responsible doctor
Arrival Number	Maintains FIFO order for equal severity

1.4 Assumptions

The following assumptions are made:

1. Every patient has a unique Patient ID.
2. Severity is represented using values from 1 to 5.
3. Higher severity means higher treatment priority.
4. Patients having equal severity follow FIFO order.
5. A patient is removed from the waiting queue after treatment.
6. One patient is assigned to one doctor at a time in the basic implementation.
7. The system operates as a simulation of an emergency department.

2. APPLICATION OF COURSE KNOWLEDGE

The project applies important concepts from Data Structures and Algorithms.

2.1 Data Structure Used

A **Priority Queue** is used to manage the patients.

A priority queue is a data structure in which elements are processed according to their priority rather than simply according to their insertion order.

In this project:

Higher Severity → Higher Priority

For equal severity:

Earlier Arrival → Higher Processing Order

2.2 Patient Structure

A C structure is used to store patient information.

Patient

└─ Patient ID
└─ Arrival Time
└─ Severity Level
└─ Assigned Doctor
└─ Arrival Number

2.3 Priority Rule

The priority comparison follows two conditions:

IF severity of Patient A > severity of Patient B

 Patient A has higher priority

ELSE IF severity is equal

 Patient with smaller arrival number has higher priority

This provides both severity-based treatment and FIFO processing.

2.4 Why Priority Queue?

Different data structures were considered.

Data Structure	Priority Handling	FIFO for Same Priority	Suitability
Simple Queue	No	Yes	Not suitable
Stack	No	No	Not suitable

Data Structure	Priority Handling	FIFO for Same Priority	Suitability
Array	Requires additional logic	Possible	Moderate
Linked List	Possible	Possible	Good
Priority Queue	Yes	Yes	Most suitable

The priority queue is selected because it directly represents the requirements of the emergency patient management problem.

3. SOLUTION / DESIGN / METHODOLOGY

3.1 Proposed Solution

The proposed system uses an **array-based priority queue**.

When a new patient arrives, the patient's details are stored in the queue. An arrival number is assigned automatically.

The system then compares patients based on:

1. Severity level.
2. Arrival number when severity is equal.

The highest-priority patient is selected for treatment.

3.2 Major Operations

Patient Insertion

A new patient is registered with Patient ID, arrival time, severity, and assigned doctor.

Priority Assignment

The severity level determines the patient's priority.

Next Patient

The system searches for the patient with the highest severity.

If two patients have the same severity, the one with the smaller arrival number is selected.

Patient Deletion

After treatment, the selected patient is removed from the waiting queue.

Searching

The system searches for a patient using Patient ID.

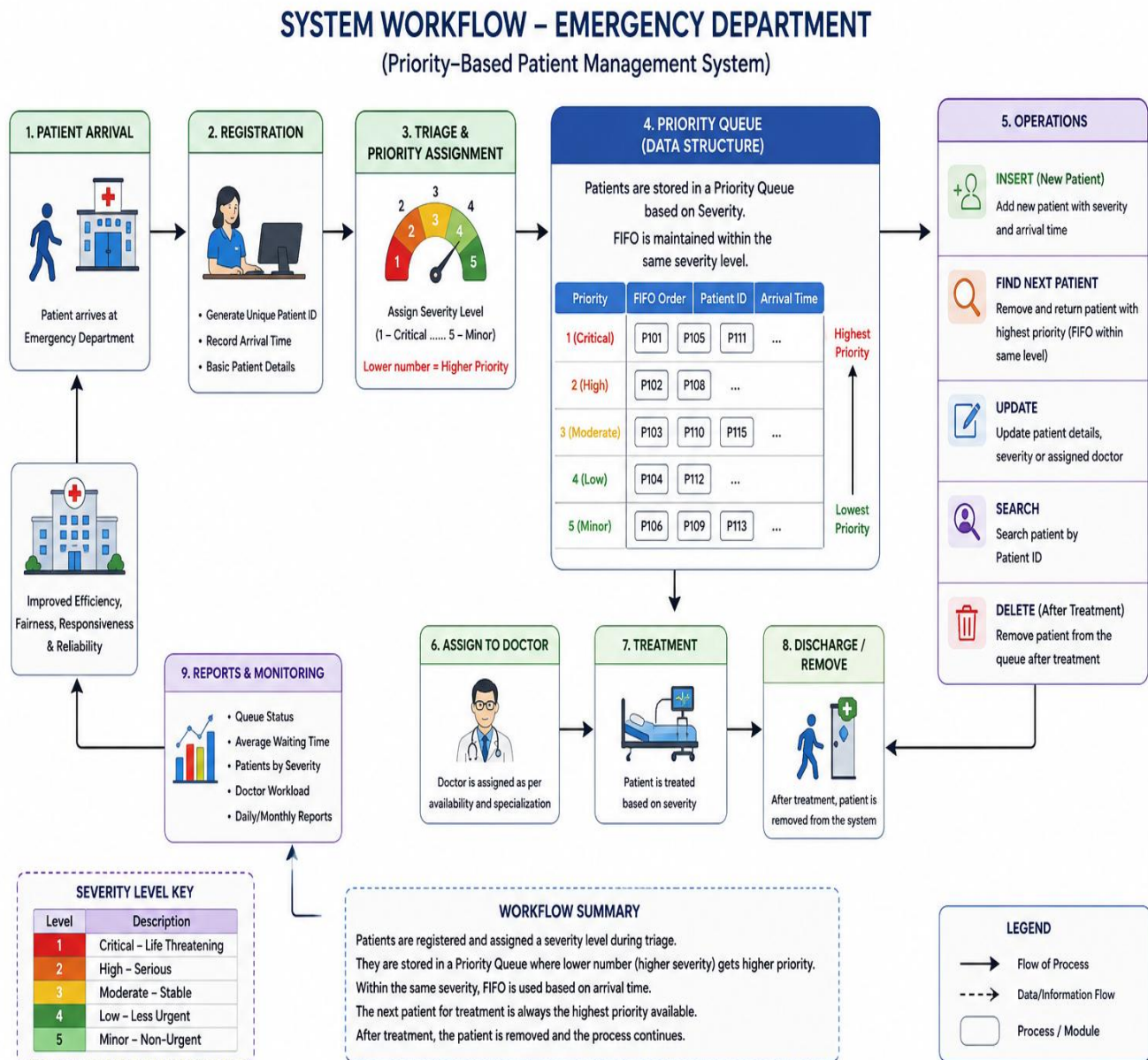
Updating

The system allows the severity, arrival time, and assigned doctor to be updated.

Display

All waiting patients are displayed according to their treatment priority.

3.3 SYSTEM WORKFLOW



4. USE OF MODERN TOOLS

The project uses the following tools:

Tool	Purpose
C Programming Language	System implementation
GCC / MinGW	Compilation
Visual Studio Code / Online C Compiler	Program development and execution

Microsoft Word	Assignment documentation
Flowchart Tool / Word Shapes	System flowchart
OnlineGDB	Optional online execution and testing

The C program is used to implement the priority queue and perform patient management operations.

5. RESULTS AND VALIDATION

The system was tested using multiple patient records and different severity levels.

Test Case 1 – Register Patient

Enter your choice: 1

Enter Patient ID: 101

Enter Arrival Time: 09:00

Enter Severity Level (1-5): 3

Enter Assigned Doctor: Dr. Kumar

Patient registered successfully!

Expected Result: Patient 101 should be registered.

Actual Result: Patient 101 was registered successfully.

Test Case 2 – Register Critical Patient

Enter your choice: 1

Enter Patient ID: 102

Enter Arrival Time: 09:05

Enter Severity Level (1-5): 5

Enter Assigned Doctor: Dr. Priya

Patient registered successfully!

Expected Result: Patient 102 should receive the highest priority.

Actual Result: Patient 102 was registered with severity 5 and received the highest priority.

Test Case 3 – Display Patient Queue

Enter your choice: 2

===== PATIENT QUEUE =====

ID	Arrival Time	Severity	Doctor	Arrival No.
102	09:05	5	Dr. Priya	2
101	09:00	3	Dr. Kumar	1

Expected Result: Patient 102 should appear before Patient 101.

Actual Result: Patient 102 appeared first because severity 5 is higher than severity 3.

Test Case 4 – Show Next Patient

Enter your choice: 3

===== NEXT PATIENT =====

Patient ID : 102

Arrival Time : 09:05

Severity : 5

Assigned Doctor: Dr. Priya

Expected Result: Patient 102 should be selected as the next patient.

Actual Result: Patient 102 was correctly identified as the next patient.

Test Case 5 – Treat Patient

Enter your choice: 4

Patient treated successfully!

Patient ID: 102

Severity: 5

Doctor: Dr. Priya

Expected Result: Patient 102 should be removed from the waiting queue.

Actual Result: Patient 102 was successfully treated and removed.

5.1 Validation of FIFO for Equal Severity

Consider:

Patient	Severity	Arrival Number
P101	5	1
P102	3	2
P103	5	3

Treatment order:

P101 → P103 → P102

P101 and P103 have the same severity. Therefore, P101 is treated first because it arrived earlier.

This confirms that the system maintains FIFO ordering within the same priority level.

6. ANALYSIS AND ENGINEERING DECISION

6.1 Time Complexity

The implemented array-based priority queue has the following complexity:

Operation	Best Case	Average Case	Worst Case
Insert	$O(1)$	$O(1)$	$O(1)$
Find Next Patient	$O(n)$	$O(n)$	$O(n)$
Delete/Treat	$O(1)$	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$	$O(n)$
Update	$O(1)$	$O(n)$	$O(n)$
Display	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(n)$	$O(n)$	$O(n)$

Here, **n** represents the number of patients currently waiting.

6.2 Explanation

Insertion

The patient is placed in the next available array position.

Best, average, and worst-case complexity:

$O(1)$

Finding Next Patient

The system may need to check all patients to identify the highest-priority patient.

Complexity:

$O(n)$

Deletion

After treatment, the remaining elements may need to be shifted.

Worst-case complexity:

$O(n)$

Searching

The system may need to examine every patient.

Best case:

$O(1)$

Worst case:

$O(n)$

Updating

The patient must first be located. Therefore, updating has a worst-case complexity of:

$O(n)$

Display

The implementation sorts a temporary array according to severity and arrival order.

Therefore:

$O(n^2)$

6.3 Limitations

The current implementation has some limitations:

1. The array has a fixed maximum capacity.
2. Searching is linear.
3. Finding the highest-priority patient requires scanning the queue.
4. Deletion may require shifting elements.
5. The system does not use a database.
6. It is a command-line application.
7. It does not automatically assign doctors based on availability.

8. It does not include real-time hospital communication.
9. It is a basic simulation and should not be considered a clinical decision-making system.

6.4 Engineering Decision

A priority queue was selected because it provides the most direct representation of the emergency treatment requirement.

A simple FIFO queue was rejected because it does not consider medical severity.

A linked list can implement priority processing, but additional logic is required.

A heap-based priority queue could provide faster priority operations for larger datasets, but the array-based priority queue is easier to understand and implement for this academic project.

7. BROADER CONSIDERATIONS

7.1 Safety

Emergency systems must ensure that critical cases are not unnecessarily delayed. The proposed priority mechanism helps ensure that patients with higher severity receive attention first.

7.2 Society

Efficient emergency patient management can contribute to better organization of healthcare services and improved responsiveness during busy periods.

7.3 Ethics

Patient information must be handled responsibly. In a real hospital system, personal and medical information should be protected from unauthorized access.

The system should also ensure that priority decisions are based on defined medical criteria rather than personal preference.

7.4 Reliability

Accurate Patient ID, severity, arrival time, and doctor information are important for reliable patient management.

7.5 Accessibility

A future version could include a graphical interface and accessibility features to make the system easier for hospital staff to use.

7.6 Professional Responsibility

This project is an academic simulation. A real emergency department system would require medical validation, security controls, testing, regulatory compliance, and integration with hospital information systems.

8. CONCLUSION

The **Priority-Based Emergency Patient Management System** provides an efficient approach for managing patients in a hospital emergency department where treatment must depend on

medical severity rather than only arrival time.

A **Priority Queue** was selected as the appropriate data structure because it supports severity-based treatment while also maintaining FIFO order among patients having the same severity. This combination is important because critically ill patients should receive attention before patients with less severe conditions, while patients with equal priority should be handled fairly according to their arrival order.

The implemented system maintains important patient information including Patient ID, arrival time, severity level, assigned doctor, and arrival number. It supports essential operations such as patient registration, priority assignment, patient display, next-patient identification, treatment and deletion, searching, and updating patient information.

The test cases demonstrate that the system correctly registers patients, identifies high-priority patients, maintains the required ordering, and removes patients after treatment. The system also correctly maintains FIFO ordering when patients have the same severity.

A conventional FIFO queue would not be appropriate for this application because it could cause a critically ill patient to wait behind a patient with a less severe condition simply because the critical patient arrived later. This could result in delayed emergency treatment. The priority queue addresses this limitation by considering both severity and arrival order.

The current array-based implementation is simple and suitable for demonstrating the data structure concepts. However, it has limitations such as fixed capacity, linear searching, and linear deletion. For a larger real-world system, a heap-based priority queue, database integration, real-time doctor availability, authentication, and secure patient records could be introduced.

Overall, the project demonstrates how selecting an appropriate data structure can solve a real-world problem effectively. The proposed priority-based system improves **efficiency, fairness, responsiveness, and reliability** in emergency patient management while demonstrating the practical application of priority queues and algorithmic decision-making.

9. STUDENT REFLECTION

9.1 What did you learn from this assignment beyond what was directly taught in the classroom?

Through this assignment, we learned how data structures can be applied to solve real-world problems. We understood that selecting a data structure depends on the requirements of the application.

We learned that a normal queue is not always suitable when different levels of priority are involved. The emergency department example helped us understand the practical importance of priority queues.

We also learned how to combine two conditions: severity-based priority and FIFO ordering for patients having equal severity.

The implementation improved our understanding of C structures, arrays, functions, searching, deletion, sorting, and time and space complexity.

We also learned the importance of testing a program using different test cases instead of checking only one normal input.

9.2 What would you improve with additional time and resources?

With additional time and resources, the system could be improved by implementing a heap-based priority queue to provide better performance for larger numbers of patients.

A database could be added to permanently store patient records. A graphical user interface could make the system easier for hospital staff to use.

The system could also include:

- Doctor availability tracking.
- Automatic doctor assignment.
- Patient waiting-time monitoring.
- Real-time priority updates.
- Secure login and authentication.
- Database backup.

10. REFERENCES

- [1] E. Horowitz, S. Sahni, and S. Anderson-Freed, *Fundamentals of Data Structures in C*. W. H. Freeman.
- [2] R. Thareja, *Data Structures Using C*. Oxford University Press.
- [3] S. Lipschutz, *Data Structures*. McGraw-Hill.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [5] H. Schildt, *C: The Complete Reference*. McGraw-Hill.
- [6] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
- [7] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *Data Structures and Algorithms*. Reading, MA, USA: Addison-Wesley, 1983.
- [8] R. L. Kruse, B. P. Leung, and C. L. Tondo, *Data Structures and Program Design in C*. Prentice Hall.

- [9] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.
- [10] Y. Langsam, M. J. Augenstein, and A. M. Tenenbaum, *Data Structures Using C and C++*, 2nd ed. Prentice Hall.
- [11] M. A. Weiss, *Data Structures and Algorithm Analysis in C*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1997.
- [12] T. H. Cormen, *Algorithms Unlocked*. Cambridge, MA, USA: MIT Press, 2013.
- [13] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Cham, Switzerland: Springer, 2020.
- [14] M. T. Goodrich and R. Tamassia, *Algorithm Design and Applications*. Hoboken, NJ, USA: Wiley, 2015.
- [15] A. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd ed. Upper Saddle River, NJ, USA: Pearson, 2007.
- [16] World Health Organization, *Emergency Care Systems for Universal Health Coverage: Ensuring Timely Care for the Acutely Ill and Injured*. Geneva, Switzerland: World Health Organization.
- [17] World Health Organization, *Basic Emergency Care: Approach to the Acutely Ill and Injured*. Geneva, Switzerland: World Health Organization.
- [18] American College of Emergency Physicians, *Emergency Department Crowding: High-Impact Solutions*. Irving, TX, USA: American College of Emergency Physicians.
- [19] G. J. Cooper, "A study of the application of computer-based systems to emergency department patient management," *Journal of Emergency Medicine*.

APPENDIX A – SOURCE CODE

```
#include <stdio.h>
```

```
#define MAX 100
```

```
struct Patient
```

```
{
```

```
    int id;
```

```
    int severity;
```

```
    int arrival;
```

```
};
```

```
struct Patient patient[MAX];
```

```
int count = 0;
```

```
int arrivalCount = 0;
```

```
/* Find the highest-priority patient */
```

```
int findPriority()
```

```
{
```

```
    int i;
```

```
    int best = 0;
```

```
    for (i = 1; i < count; i++)
```

```
    {
```

```
        /* Lower severity number = higher priority */
```

```
        if (patient[i].severity < patient[best].severity)
```

```
        {
```

```
            best = i;
```

```
        }
```

```
        /* Same severity -> FIFO */
```

```
        else if (patient[i].severity == patient[best].severity &&
```

```
                patient[i].arrival < patient[best].arrival)
```

```
        {
```

```
            best = i;
```

```
        }
```

```
    }
```

```

    return best;
}

/* Register a new patient */
void addPatient()
{
    if (count == MAX)
    {
        printf("\nQueue is full!\n");
        return;
    }

    printf("\nEnter Patient ID: ");
    scanf("%d", &patient[count].id);

    printf("Enter Severity Level (1-5): ");
    scanf("%d", &patient[count].severity);

    if (patient[count].severity < 1 ||
        patient[count].severity > 5)
    {
        printf("Invalid severity level!\n");
        return;
    }

    arrivalCount++;
    patient[count].arrival = arrivalCount;

    count++;

```

```
printf("\nPatient registered successfully!\n");  
}
```

```
/* Display patients according to priority */
```

```
void display()
```

```
{
```

```
    int i, j;
```

```
    struct Patient temp;
```

```
    if (count == 0)
```

```
    {
```

```
        printf("\nNo patients waiting.\n");
```

```
        return;
```

```
    }
```

```
/* Sort according to severity and FIFO */
```

```
for (i = 0; i < count - 1; i++)
```

```
{
```

```
    for (j = i + 1; j < count; j++)
```

```
    {
```

```
        if (patient[j].severity < patient[i].severity ||
```

```
            (patient[j].severity == patient[i].severity &&
```

```
            patient[j].arrival < patient[i].arrival))
```

```
        {
```

```
            temp = patient[i];
```

```
            patient[i] = patient[j];
```

```
            patient[j] = temp;
```

```
        }
```

```

    }
}

printf("\n===== PATIENT QUEUE =====\n");

printf("ID\tSeverity\tArrival No.\n");

for (i = 0; i < count; i++)
{
    printf("%d\t%d\t%d\n",
           patient[i].id,
           patient[i].severity,
           patient[i].arrival);
}
}

/* Show the next patient */
void nextPatient()
{
    int index;

    if (count == 0)
    {
        printf("\nNo patients waiting.\n");
        return;
    }

    index = findPriority();

```

```

printf("\n===== NEXT PATIENT =====\n");

printf("Patient ID : %d\n", patient[index].id);
printf("Severity : %d\n", patient[index].severity);
printf("Arrival No.: %d\n", patient[index].arrival);
}

/* Treat and remove the highest-priority patient */
void treatPatient()
{
    int index;
    int i;

    if (count == 0)
    {
        printf("\nNo patients waiting.\n");
        return;
    }

    index = findPriority();

    printf("\nPatient treated successfully!\n");

    printf("Patient ID: %d\n", patient[index].id);
    printf("Severity: %d\n", patient[index].severity);

    /* Remove patient */
    for (i = index; i < count - 1; i++)
    {

```

```

        patient[i] = patient[i + 1];
    }

    count--;
}

/* Search patient */
void searchPatient()
{
    int id;
    int i;

    printf("\nEnter Patient ID to search: ");
    scanf("%d", &id);

    for (i = 0; i < count; i++)
    {
        if (patient[i].id == id)
        {
            printf("\nPatient Found!\n");

            printf("Patient ID : %d\n", patient[i].id);
            printf("Severity   : %d\n", patient[i].severity);
            printf("Arrival No.: %d\n", patient[i].arrival);

            return;
        }
    }
}

```

```
printf("\nPatient not found.\n");
}

/* Update patient information */
void updatePatient()
{
    int id;
    int i;

    printf("\nEnter Patient ID to update: ");
    scanf("%d", &id);

    for (i = 0; i < count; i++)
    {
        if (patient[i].id == id)
        {
            printf("Enter New Severity Level (1-5): ");
            scanf("%d", &patient[i].severity);

            if (patient[i].severity < 1 ||
                patient[i].severity > 5)
            {
                printf("Invalid severity level!\n");
                return;
            }

            printf("\nPatient information updated successfully!\n");

            return;
        }
    }
}
```



```

    }

}

printf("\nPatient not found.\n");
}

/* Main function */
int main()
{
    int choice;

    do
    {
        printf("\n\n===== \n");
        printf("  EMERGENCY PATIENT MANAGEMENT SYSTEM\n");
        printf("===== \n");

        printf("1. Register New Patient\n");
        printf("2. Display Patient Queue\n");
        printf("3. Show Next Patient\n");
        printf("4. Treat Patient\n");
        printf("5. Search Patient\n");
        printf("6. Update Patient\n");
        printf("7. Exit\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)

```

```
{  
    case 1:  
        addPatient();  
        break;  
  
    case 2:  
        display();  
        break;  
  
    case 3:  
        nextPatient();  
        break;  
  
    case 4:  
        treatPatient();  
        break;  
  
    case 5:  
        searchPatient();  
        break;  
  
    case 6:  
        updatePatient();  
        break;  
  
    case 7:  
        printf("\nExiting system...\n");  
        break;  
  
    default:
```

```

        printf("\nInvalid choice! Try again.\n");
    }

    } while (choice != 7);

    return 0;
}

```

APPENDIX B – EXECUTION SCREENSHOTS

The following screenshots should be captured after running the program and inserted into the report:

Screenshot 1: Main Menu

Screenshot 2: Registering Patients

Screenshot 3: Displaying Patient Queue

Screenshot 4: Showing Next Patient

Screenshot 5: Treating a Patient

Screenshot 6: Searching for a Patient

Screenshot 7: Updating Patient Information

APPENDIX C – GROUP MEMBERS

Member 1 – G.Ramya

- Analyzed the problem statement and requirements.
- Studied suitable data structures.
- Designed the priority queue.
- Developed patient registration and priority handling.
- Implemented next-patient selection and treatment operations.

Member 2 – R.Varshini

- Implemented search and update operations.
- Prepared the algorithm, pseudocode, and flowchart.
- Prepared and tested test cases.
- Verified program outputs.

- Performed complexity analysis.
- Prepared documentation and presentation material.

APPENDIX D – TESTING SUMMARY

Test Case	Operation	Expected Result	Status
1	Register Patient	Patient registered	Passed
2	Register Critical Patient	Highest priority assigned	Passed
3	Display Queue	Highest severity displayed first	Passed
4	Show Next Patient	Critical patient selected	Passed
5	Treat Patient	Selected patient removed	Passed
6	Search Patient	Correct patient details displayed	Passed
7	Update Patient	Information updated	Passed
8	Same Severity	Earlier patient selected first	Passed

FINAL PROJECT SUMMARY

The project successfully demonstrates the application of a **Priority Queue** to emergency patient management. The system gives priority to patients according to severity and maintains FIFO order among patients with equal severity. The implementation demonstrates important data-structure operations including insertion, searching, updating, priority selection, and deletion.

The solution satisfies the main requirements of the problem and demonstrates how appropriate data-structure selection can improve the management of real-world systems.

One Page Write Up:

The Emergency Patient Management System was implemented and tested to evaluate its ability to manage patients according to their medical severity and arrival order. The system maintains important patient information such as Patient ID, severity level, and arrival number. A priority-based approach was used, where Severity Level 1 represents the highest priority (critical condition) and Severity Level 5 represents the lowest priority. When two or more patients have the same severity level, the system follows the First-In-First-Out (FIFO) principle using the patient's arrival number.

Several test cases were performed to validate the major operations of the system. In the first test case, Patient 101 was registered with severity level 3. The system successfully stored the patient and assigned an arrival number. In the second test case, Patient 102 was registered with severity level 1. Since severity level 1 has the highest priority, the system correctly identified Patient 102 as more urgent than Patient 101. The display operation was then tested, and Patient 102 appeared before Patient 101 in the queue. This confirmed that patients are arranged according to their priority rather than simply according to their arrival time.

The next-patient operation was tested to verify whether the system could identify the patient who should receive treatment first. Patient 102 was correctly selected because of the critical severity level. The treatment operation was then performed, and Patient 102 was successfully treated and removed from the waiting queue. This demonstrated that the system can efficiently select and delete the highest-priority patient.

The search operation was tested using Patient ID 101. The system successfully located the patient and displayed the stored details. The update operation was also verified by changing Patient 101's severity level from 3 to 2. The system accepted the new value and displayed the updated information correctly. This confirms that patient information can be modified whenever the patient's condition changes.

Finally, the FIFO rule for patients with equal severity was tested. Patients 103 and 104 were registered with severity level 2. Patient 101 also had severity level 2 after the update. Although all three patients had the same priority, the system maintained their arrival order. Patient 101, having the earliest arrival number, was selected first, followed by Patient 103 and Patient 104. Thus, the system successfully implemented priority-based treatment with FIFO ordering within the same severity level.

The overall results show that all major operations produced the expected outcomes. The system successfully handles patient registration, priority assignment, queue display, next-patient selection, treatment and deletion, searching, updating, and FIFO processing. The testing confirms that the selected priority-based approach is more appropriate than a conventional FIFO queue for an emergency department. It prevents critically ill patients from waiting behind less severe cases while maintaining fairness among patients with equal priority. Therefore, the proposed system improves the efficiency, responsiveness, fairness, and reliability of emergency patient management.

A.Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

C. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action